JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class Executable

java.lang.Object
    java.lang.reflect.AccessibleObject
        java.lang.reflect.Executable

All Implemented Interfaces:

AnnotatedElement, GenericDeclaration, Member

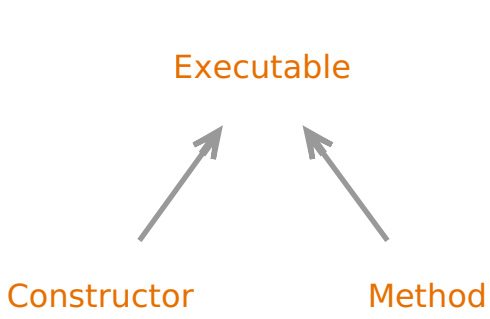
Direct Known Subclasses:

Constructor, Method

```
public abstract sealed class Executable
extends AccessibleObject
implements Member, GenericDeclaration
permits Constructor<T>, Method
```

A shared superclass for the common functionality of Method and Constructor.

Sealed Class Hierarchy Graph:



Since:

1.8

Field Summary

Fields declared in interface Member
DECLARED, PUBLIC

Method Summary

All Methods	Instance Methods	
Modifier and Type	**Method**	**Description**
Set<AccessFlag>	**accessFlags**()	Returns an unmodifiable set of the access flags for the executable

		represented by this object, possibly empty.
AnnotatedType[]	**getAnnotatedExceptionTyp**	Returns an array of AnnotatedType objects that represent the use of types to specify the declared exceptions of the method/constructor represented by this Executable.
AnnotatedType[]	**getAnnotatedParameterTyp**	Returns an array of AnnotatedType objects that represent the use of types to specify formal parameter types of the method/constructor represented by this Executable.
AnnotatedType	**getAnnotatedReceiverType**	Returns an AnnotatedType object that represents the use of a type to specify the receiver type of the method/constructor represented by this Executable object.
abstract AnnotatedType	**getAnnotatedReturnType**()	Returns an AnnotatedType object that represents the use of a type to specify the return type of the method/constructor represented by this Executable.
<T extends Annotation> T	**getAnnotation** (Class<T> annotationClas	Returns this element's annotation for the specified type if such an

		annotation is *present*, else null.
<T extends Annotation> T[]	**getAnnotationsByType**(Class<T> annotationClas	Returns annotations that are *associated* with this element.
abstract Class<?>	**getDeclaringClass**()	Returns the Class object representing the class or interface that declares the executable represented by this object.
abstract Class<?>[]	**getExceptionTypes**()	Returns an array of Class objects that represent the types of exceptions declared to be thrown by the underlying executable represented by this object.
Type[]	**getGenericExceptionTypes**	Returns an array of Type objects that represent the exceptions declared to be thrown by this executable object.
Type[]	**getGenericParameterTypes**	Returns an array of Type objects that represent the formal parameter types, in declaration order, of the executable represented by this object.
abstract int	**getModifiers**()	Returns the Java language modifiers for the executable represented by this object.

abstract String	**getName**()	Returns the name of the executable represented by this object.
abstract Annotation[][]	**getParameterAnnotations**(	Returns an array of arrays of Annotations that represent the annotations on the formal parameters, in declaration order, of the Executable represented by this object.
abstract int	**getParameterCount**()	Returns the number of formal parameters (whether explicitly declared or implicitly declared or neither) for the executable represented by this object.
Parameter[]	**getParameters**()	Returns an array of Parameter objects representing all the parameters to the underlying executable represented by this object.
abstract Class<?>[]	**getParameterTypes**()	Returns an array of Class objects that represent the formal parameter types, in declaration order, of the executable represented by this object.
abstract TypeVariable<?>[]	**getTypeParameters**()	Returns an array of TypeVariable objects that represent the type variables declared by the generic declaration represented by this GenericDeclaration

		object, in declaration order.
boolean	**isSynthetic**()	Returns `true` if this executable is a synthetic construct; returns `false` otherwise.
boolean	**isVarArgs**()	Returns `true` if this executable was declared to take a variable number of arguments; returns `false` otherwise.
String	**toGenericString**()	Returns a string describing this `Executable`, including any type parameters.

Methods declared in class AccessibleObject

canAccess, getAnnotations, getDeclaredAnnotation, getDeclaredAnnotations, getDeclaredAnnotationsByType, isAccessible, isAnnotationPresent, setAccessible, setAccessible, trySetAccessible

Methods declared in class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Method Details

getDeclaringClass

```
public abstract Class<?> getDeclaringClass()
```

Returns the `Class` object representing the class or interface that declares the executable represented by this object.

Specified by:

`getDeclaringClass` in interface `Member`

Returns:

an object representing the declaring class of the underlying member

getName

```
public abstract String getName()
```

Returns the name of the executable represented by this object.

Specified by:

`getName` in interface `Member`

Returns:

the simple name of the underlying member

getModifiers

```
public abstract int getModifiers()
```

Returns the Java language modifiers for the executable represented by this object.

Specified by:

`getModifiers` in interface `Member`

Returns:

the Java language modifiers for the executable represented by this object

See Also:

`accessFlags()`

accessFlags

```
public Set<AccessFlag> accessFlags()
```

Returns an unmodifiable set of the access flags for the executable represented by this object, possibly empty.

Specified by:

`accessFlags` in interface `Member`

Returns:

an unmodifiable set of the access flags for the executable represented by this object, possibly empty

See *Java Virtual Machine Specification*:

4.6 Methods

Since:

20

See Also:

`getModifiers()`

getTypeParameters

```
public abstract TypeVariable<?>[] getTypeParameters()
```

Returns an array of `TypeVariable` objects that represent the type variables declared by the generic declaration represented by this `GenericDeclaration` object, in declaration order. Returns an array of length 0 if the underlying generic declaration declares no type variables.

Specified by:

`getTypeParameters` in interface `GenericDeclaration`

Returns:

an array of `TypeVariable` objects that represent the type variables declared by this generic declaration

Throws:

`GenericSignatureFormatError` - if the generic signature of this generic declaration does not conform to the format specified in *The Java Virtual Machine Specification*

getParameterTypes

```
public abstract Class<?>[] getParameterTypes()
```

Returns an array of `Class` objects that represent the formal parameter types, in declaration order, of the executable represented by this object. Returns an array of length 0 if the underlying executable takes no parameters. Note that the constructors of some inner classes may have an implicitly declared parameter in addition to explicitly declared ones. Also note that compact constructors of a record class may have implicitly declared parameters.

Returns:

the parameter types for the executable this object represents

getParameterCount

```
public abstract int getParameterCount()
```

Returns the number of formal parameters (whether explicitly declared or implicitly declared or neither) for the executable represented by this object.

Returns:

The number of formal parameters for the executable this object represents

getGenericParameterTypes

```
public Type[] getGenericParameterTypes()
```

Returns an array of `Type` objects that represent the formal parameter types, in declaration order, of the executable represented by this object. An array of length 0 is returned if the underlying executable takes no parameters. Note that the constructors of some inner classes may have an implicitly declared parameter in addition to explicitly declared ones. Compact constructors of a record class may also have implicitly declared parameters, but they are a special case and thus considered as if they had been explicitly declared in the source. Finally note that as a `modeling artifact`, the number of returned parameters can differ depending on whether or not generic information is present. If generic information is present, parameters explicitly present in the source or parameters of compact constructors of a record class will be returned. Note that parameters of compact constructors of a record class are a special case, as they are not explicitly present in the source, and its type will be returned regardless of the parameters being implicitly declared or not. If generic information is not present, implicit and synthetic parameters may be returned as well.

If a formal parameter type is a parameterized type, the `Type` object returned for it must accurately reflect the actual type arguments used in the source code. This assertion also applies to the parameters of compact constructors of a record class, independently of them being implicitly declared or not.

If a formal parameter type is a type variable or a parameterized type, it is created. Otherwise, it is resolved.

Returns:

an array of `Type`s that represent the formal parameter types of the underlying executable, in declaration order

Throws:

`GenericSignatureFormatError` - if the generic method signature does not conform to the format specified in *The Java Virtual Machine Specification*

`TypeNotPresentException` - if any of the parameter types of the underlying executable refers to a non-existent type declaration

`MalformedParameterizedTypeException` - if any of the underlying executable's parameter types refer to a parameterized type that cannot be instantiated for any reason

getParameters

```
public Parameter[] getParameters()
```

Returns an array of `Parameter` objects representing all the parameters to the underlying executable represented by this object. An array of length 0 is returned if the executable has no parameters.

The parameters of the underlying executable do not necessarily have unique names, or names that are legal identifiers in the Java programming language (JLS §3.8).

Returns:

an array of `Parameter` objects representing all the parameters to the underlying executable represented by this object

Throws:

`MalformedParametersException` - if the class file contains a MethodParameters attribute that is improperly formatted.

getExceptionTypes

```
public abstract Class<?>[] getExceptionTypes()
```

Returns an array of `Class` objects that represent the types of exceptions declared to be thrown by the underlying executable represented by this object. Returns an array of length 0 if the executable declares no exceptions in its `throws` clause.

Returns:

the exception types declared as being thrown by the executable this object represents

getGenericExceptionTypes

```
public Type[] getGenericExceptionTypes()
```

Returns an array of `Type` objects that represent the exceptions declared to be thrown by this executable object. Returns an array of length 0 if the underlying executable declares no exceptions in its `throws` clause.

If an exception type is a type variable or a parameterized type, it is created. Otherwise, it is resolved.

Returns:

an array of Types that represent the exception types thrown by the underlying executable

Throws:

`GenericSignatureFormatError` - if the generic method signature does not conform to the format specified in *The Java Virtual Machine Specification*

`TypeNotPresentException` - if the underlying executable's `throws` clause refers to a non-existent type declaration

`MalformedParameterizedTypeException` - if the underlying executable's `throws` clause refers to a parameterized type that cannot be instantiated for any reason

toGenericString

```
public String toGenericString()
```

Returns a string describing this `Executable`, including any type parameters.

Returns:

a string describing this `Executable`, including any type parameters

isVarArgs

```
public boolean isVarArgs()
```

Returns `true` if this executable was declared to take a variable number of arguments; returns `false` otherwise.

Returns:

`true` if this executable was declared to take a variable number of arguments; returns `false` otherwise

isSynthetic

```
public boolean isSynthetic()
```

Returns `true` if this executable is a synthetic construct; returns `false` otherwise.

Specified by:

`isSynthetic` in interface `Member`

Returns:

true if and only if this executable is a synthetic construct as defined by *The Java Language Specification*.

See *Java Language Specification*:

13.1 The Form of a Binary

See *Java Virtual Machine Specification*:

4.6 Methods

getParameterAnnotations

```
public abstract Annotation[][] getParameterAnnotations()
```

Returns an array of arrays of `Annotations` that represent the annotations on the formal parameters, in declaration order, of the `Executable` represented by this object. Synthetic and mandated parameters (see explanation below), such as the outer "this" parameter to an inner class constructor will be represented in the returned array. If the executable has no parameters (meaning no formal, no synthetic, and no mandated parameters), a zero-length array will be returned. If the `Executable` has one or more parameters, a nested array of length zero is returned for each parameter with no annotations. The annotation objects contained in the returned arrays are serializable. The caller of this method is free to modify the returned arrays; it will have no effect on the arrays returned to other callers. A compiler may add extra parameters that are implicitly declared in source ("mandated"), as well as parameters that are neither implicitly nor explicitly declared in source ("synthetic") to the parameter list for a method. See `Parameter` for more information.

Note that any annotations returned by this method are declaration annotations.

Returns:

an array of arrays that represent the annotations on the formal and implicit parameters, in declaration order, of the executable represented by this object

See Also:

`Parameter`, `Parameter.getAnnotations()`

getAnnotation

```
public <T extends Annotation> T getAnnotation
(Class<T> annotationClass)
```

Returns this element's annotation for the specified type if such an annotation is *present*, else null.

Note that any annotation returned by this method is a declaration annotation.

Specified by:

`getAnnotation` in interface `AnnotatedElement`

Overrides:

`getAnnotation` in class `AccessibleObject`

Type Parameters:

T - the type of the annotation to query for and return if present

Parameters:

`annotationClass` - the Class object corresponding to the annotation type

Returns:

this element's annotation for the specified annotation type if present on this element, else null

Throws:

`NullPointerException` - if the given annotation class is null

getAnnotationsByType

```
public <T extends Annotation> T[] getAnnotationsByType
(Class<T> annotationClass)
```

Returns annotations that are *associated* with this element. If there are no annotations *associated* with this element, the return value is an array of length 0. The difference between this method and `AnnotatedElement.getAnnotation(Class)` is that this method detects if its argument is a *repeatable annotation type* (JLS §9.6), and if so, attempts to find one or more annotations of that type by "looking through" a container annotation. The caller of this method is free to modify the returned array; it will have no effect on the arrays returned to other callers.

Note that any annotations returned by this method are declaration annotations.

Specified by:

`getAnnotationsByType` in interface `AnnotatedElement`

Overrides:

`getAnnotationsByType` in class `AccessibleObject`

Type Parameters:

T - the type of the annotation to query for and return if present

Parameters:

`annotationClass` - the Class object corresponding to the annotation type

Returns:

all this element's annotations for the specified annotation type if associated with this element, else an array of length zero

Throws:

`NullPointerException` - if the given annotation class is null

getAnnotatedReturnType

```
public abstract AnnotatedType getAnnotatedReturnType()
```

Returns an `AnnotatedType` object that represents the use of a type to specify the return type of the method/constructor represented by this Executable. If this `Executable` object represents a constructor, the `AnnotatedType` object represents the type of the constructed object. If this `Executable` object represents a method, the `AnnotatedType` object represents the use of a type to specify the return type of the method.

Returns:

an object representing the return type of the method or constructor represented by this `Executable`

getAnnotatedReceiverType

```
public AnnotatedType getAnnotatedReceiverType()
```

Returns an `AnnotatedType` object that represents the use of a type to specify the receiver type of the method/constructor represented by this `Executable` object. The receiver type of a method/constructor is available only if the method/constructor has a receiver parameter (JLS §8.4.1). If this `Executable` object *represents an instance method or represents a constructor of an inner member class*, and the method/constructor *either* has no receiver parameter or has a receiver parameter with no annotations on its type, then the return value is an `AnnotatedType` object representing an element with no annotations. If this `Executable` object represents a static method or represents a constructor of a top level, static member, local, or anonymous class, then the return value is null.

Returns:

an object representing the receiver type of the method or constructor represented by this `Executable` or `null` if this `Executable` can not have a receiver parameter

See *Java Language Specification*:

8.4 Method Declarations
8.4.1 Formal Parameters
8.8 Constructor Declarations

getAnnotatedParameterTypes

```
public AnnotatedType[] getAnnotatedParameterTypes()
```

Returns an array of `AnnotatedType` objects that represent the use of types to specify formal parameter types of the method/constructor represented by this Executable. The order of the objects in the array corresponds to the order of

the formal parameter types in the declaration of the method/constructor. Returns an array of length 0 if the method/constructor declares no parameters. Note that the constructors of some inner classes may have an implicitly declared parameter in addition to explicitly declared ones. Also note that compact constructors of a record class may have implicitly declared parameters.

Returns:

an array of objects representing the types of the formal parameters of the method or constructor represented by this `Executable`

getAnnotatedExceptionTypes

```
public AnnotatedType[] getAnnotatedExceptionTypes()
```

Returns an array of `AnnotatedType` objects that represent the use of types to specify the declared exceptions of the method/constructor represented by this Executable. The order of the objects in the array corresponds to the order of the exception types in the declaration of the method/constructor. Returns an array of length 0 if the method/constructor declares no exceptions.

Returns:

an array of objects representing the declared exceptions of the method or constructor represented by this `Executable`

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.